

Complete List of Java Keywords — Java 1.0 → Java 26

Java keywords are reserved words defined by the **Java Language Specification (JLS)**. They have predefined syntactic/semantic meaning and, in general, cannot be used as identifiers such as class names, method names, variable names, or package names.

As of **Java SE 26**, the JLS defines **51 reserved keywords** and **17 contextual keywords**. true, false, and null are **literals, not keywords**. ([Oracle Docs](#))

1. Java Keywords — Complete Classification

Category	Count in Java 26	Keywords
Reserved keywords	51	abstract, assert, boolean, break, byte, case, catch, char, class, const, continue, default, do, double, else, enum, extends, final, finally, float, for, goto, if, implements, import, instanceof, int, interface, long, native, new, package, private, protected, public, return, short, static, strictfp, super, switch, synchronized, this, throw, throws, transient, try, void, volatile, while, _
Contextual keywords	17	exports, opens, requires, uses, yield, module, permits, sealed, var, non-sealed, provides, to, when, open, record, transitive, with
Boolean literals	2	true, false
Null literal	1	null

The important distinction is that **reserved keywords are always treated as keywords**, whereas **contextual keywords acquire keyword meaning only in particular grammatical contexts**. ([Oracle Docs](#))

2. Complete Reserved Keyword List

The following 51 are reserved by Java 26:

#	Keyword	Category	Meaning / Purpose	Example
1	abstract	Class/Method modifier	Declares an incomplete class or method that must be implemented by a subclass/concrete class.	abstract class Animal { abstract void sound(); }
2	assert	Assertion	Tests an assumption during program execution.	assert age >= 18;
3	boolean	Primitive type	Represents true or false.	boolean active = true;
4	break	Control flow	Terminates a loop or switch statement.	if (x == 5) break;
5	byte	Primitive type	8-bit signed integer type.	byte b = 10;
6	case	switch	Defines a possible matching branch in switch.	case 1: System.out.println("One");
7	catch	Exception handling	Handles an exception thrown from a try block.	catch (Exception e) { }
8	char	Primitive type	Represents a UTF-16 code unit.	char ch = 'A';
9	class	Type declaration	Declares a class.	class Student { }
10	const	Reserved/unused	Reserved for possible future use; Java does not use it for constants.	// not used in Java
11	continue	Control flow	Skips the current loop iteration and continues with the next iteration.	if (x == 5) continue;

12	default	switch / interface	Specifies a default switch branch or a default interface method.	default: System.out.println("Other");
13	do	Loop	Starts a do-while loop.	do { x++; } while (x < 10);
14	double	Primitive type	64-bit floating-point type.	double price = 99.50;
15	else	Conditional	Specifies the alternative branch of an if.	if (x > 0) {} else {}
16	enum	Type declaration	Declares an enumeration type.	enum Day { MON, TUE, WED }
17	extends	Inheritance	Specifies class inheritance or interface inheritance.	class Dog extends Animal { }
18	final	Modifier	Prevents reassignment, overriding, or inheritance depending on where it is used.	final int MAX = 100;
19	finally	Exception handling	Defines a block normally executed after try/catch.	finally { close(); }
20	float	Primitive type	32-bit floating-point type.	float f = 10.5f;
21	for	Loop	Creates a traditional for loop or enhanced for loop.	for (int i=0; i<10; i++) { }
22	goto	Reserved/unused	Reserved but not implemented in Java.	// not used in Java
23	if	Conditional	Executes code conditionally.	if (age >= 18) { }
24	implements	Interface implementation	Indicates that a class implements one or more interfaces.	class Dog implements Animal { }
25	import	Compilation/package	Makes types or static members available by simple name.	import java.util.List;

26	instanceof	Type testing	Tests whether an object is compatible with a specified type/pattern.	if (obj instanceof String s) {}
27	int	Primitive type	32-bit signed integer type.	int age = 25;
28	interface	Type declaration	Declares an interface.	interface Vehicle { void start(); }
29	long	Primitive type	64-bit signed integer type.	long id = 100000L;
30	native	Method modifier	Indicates that a method is implemented in native code, typically through JNI.	native void execute();
31	new	Object creation	Creates an object, array, or certain other runtime entities.	Student s = new Student();
32	package	Namespace	Declares the package to which a compilation unit belongs.	package com.example.app;
33	private	Access modifier	Makes a member accessible only within its declaring top-level class or interface context.	private int age;
34	protected	Access modifier	Provides access to members from the same package and appropriate subclasses.	protected int id;
35	public	Access modifier	Makes a top-level type or member broadly accessible subject to Java's access rules.	public class Student {}
36	return	Control flow	Returns control, optionally with a value, from a method or lambda.	return total;
37	short	Primitive type	16-bit signed integer type.	short s = 100;

38	static	Class-level modifier	Associates a member with the class rather than individual instances.	static int count;
39	strictfp	Floating-point modifier	Historically required strict floating-point evaluation. It is obsolete in modern Java and should not be used in new code.	strictfp class MathUtil {}
40	super	Inheritance	Refers to a superclass member or invokes a superclass constructor.	super.display();
41	switch	Control flow	Selects among multiple execution paths.	switch (day) { case 1 -> ...; }
42	synchronized	Concurrency	Controls access to a block/method using an object's or class's monitor.	synchronized void update() {}
43	this	Object reference	Refers to the current object.	this.name = name;
44	throw	Exception handling	Explicitly throws an exception.	throw new IllegalArgumentException();
45	throws	Exception declaration	Declares checked exceptions that a method/constructor may throw.	void read() throws IOException {}
46	transient	Serialization	Prevents an instance field from being serialized by default Java serialization.	transient String password;
47	try	Exception handling	Defines code whose exceptions can be caught/finalized.	try { read(); } catch (...) {}
48	void	Return type	Indicates that a method does not return a value.	void display() {}
49	volatile	Concurrency	Gives a field special memory-visibility/ordering semantics for concurrent access.	volatile boolean running;

50	while	Loop	Repeatedly executes a statement while a condition is true.	while (x < 10) x++;
51	_	Reserved keyword	A standalone underscore is reserved and cannot be used as an ordinary identifier; Java 26 permits it only in certain declaration contexts.	int _ = 10;*

* The underscore has special modern rules; it should **not** be treated as a normal variable name. The JLS specifically lists _ among the reserved keywords. ([Oracle Docs](#))

3. Contextual Keywords in Java 26

Contextual keywords are different from traditional reserved keywords. Their interpretation depends on where they occur in the source code. Java adopted this approach particularly to introduce newer language features while preserving source compatibility with older programs. ([Oracle Docs](#))

#	Contextual keyword	Introduced / associated feature	Purpose	Example
1	module	Java 9	Declares a module.	module com.example.app { }
2	open	Java 9	Declares an open module.	open module com.example.app { }
3	requires	Java 9	Declares a module dependency.	requires java.sql;
4	transitive	Java 9	Makes a module dependency transitively readable.	requires transitive com.api;
5	exports	Java 9	Exports a package from a module.	exports com.example.api;
6	opens	Java 9	Opens a package for reflective access.	opens com.example.model;

7	to	Java 9	Restricts an exports/opens directive to named modules.	exports com.api to client;
8	uses	Java 9	Declares a service used by a module.	uses com.example.Service;
9	provides	Java 9	Declares a service implementation.	provides Service with ServiceImpl;
10	with	Java 9	Specifies implementation classes for a service.	provides S with SImpl;
11	var	Java 10	Enables local-variable type inference.	var name = "Anurag";
12	yield	Java 13/14	Returns a value from a switch expression.	yield 100;
13	record	Java 14/16	Declares a record class.	record Student(int id, String name) {}
14	sealed	Java 15/17	Restricts which classes/interfaces can directly extend/implement a type.	sealed class Shape permits Circle {}
15	non-sealed	Java 15/17	Removes the sealing restriction for a permitted subclass.	non-sealed class Circle extends Shape {}
16	permits	Java 15/17	Specifies permitted direct subclasses/subinterfaces of a sealed type.	sealed class Shape permits Circle {}
17	when	Modern pattern matching	Provides a guard condition in applicable pattern matching constructs.	case String s when !s.isEmpty() -> ...

Java 26 currently specifies these **17 contextual keywords**. ([Oracle Docs](#))

4. Java Keywords by Functional Category

Another useful way to study keywords is by **what they do** rather than alphabetically.

A. Data Type Keywords

Keyword	Type	Description	Example
boolean	Primitive	Boolean value	boolean valid = true;
byte	Primitive	8-bit signed integer	byte b = 10;
char	Primitive	UTF-16 code unit	char c = 'A';
short	Primitive	16-bit signed integer	short s = 100;
int	Primitive	32-bit signed integer	int n = 1000;
long	Primitive	64-bit signed integer	long n = 1000L;
float	Primitive	32-bit floating point	float f = 10.5f;
double	Primitive	64-bit floating point	double d = 10.5;
void	Method return type	No return value	void print() {}

5. Access Modifier Keywords

Keyword	Access	Typical use	Example
public	Broadest accessibility	Public API	public class Employee {}

protected	Package + subclass access	Inheritance	protected int salary;
private	Declaring class/interface context	Encapsulation	private int id;

There is **no internal keyword** in Java like there is in some other languages.

6. Non-Access Modifier Keywords

Keyword	Main purpose	Example
static	Class-level member	static int count;
final	Prevent modification/extension/overriding	final int MAX = 10;
abstract	Incomplete class/method	abstract void run();
synchronized	Monitor-based synchronization	synchronized void update() {}
volatile	Visibility/ordering semantics for a field	volatile boolean running;
transient	Exclude field from default serialization	transient String password;
native	Native implementation	native void execute();
strictfp	Historical strict FP semantics; obsolete	strictfp class A {}
sealed	Restrict inheritance	sealed class A permits B {}
non-sealed	Allow unrestricted subclassing from a sealed hierarchy	non-sealed class B extends A {}

7. Class and Object Keywords

Keyword	Purpose	Example
class	Declares a class	class Employee {}
interface	Declares an interface	interface Printable {}
enum	Declares an enum	enum Status { NEW, DONE }
record	Declares a record	record Employee(int id) {}
abstract	Declares abstract type/member	abstract class Shape {}
final	Prevents inheritance	final class StringUtil {}
sealed	Restricts permitted inheritance	sealed class Shape permits Circle {}
non-sealed	Removes sealing restriction for a permitted subtype	non-sealed class Circle extends Shape {}
permits	Lists permitted subclasses	sealed class Shape permits Circle {}
extends	Inherits from class/interface	class Dog extends Animal {}
implements	Implements interface(s)	class Dog implements Animal {}
new	Creates objects/arrays	new Employee()
this	Current object	this.id = id;
super	Superclass reference	super.display();

8. Control-Flow Keywords

Keyword	Purpose	Example
if	Conditional execution	if (x > 0) {}
else	Alternative branch	else {}
switch	Multi-way selection	switch (x) {}
case	Switch branch	case 1 -> ...
default	Default switch branch	default -> ...
for	Loop	for (int i=0; i<10; i++)
while	Conditional loop	while (x < 10)
do	Post-test loop	do {} while (x < 10);
break	Exit loop/switch	break;
continue	Skip current iteration	continue;
return	Exit method/lambda	return result;
yield	Produce value from switch expression	yield result;

9. Exception-Handling Keywords

Keyword	Purpose	Example
try	Defines exception-sensitive code	try { read(); }

catch	Handles exception	catch (IOException e) {}
finally	Cleanup/final execution block	finally { close(); }
throw	Explicitly throws exception	throw new RuntimeException();
throws	Declares exceptions	void read() throws IOException {}

throw vs throws

Aspect	throw	throws
Purpose	Actually throws an exception	Declares possible exceptions
Location	Method body	Method/constructor declaration
Number	One exception object at a time	Can declare multiple exception types
Example	throw new IOException();	void read() throws IOException

10. Package and Module Keywords

Java 9 introduced the **Java Platform Module System (JPMS)**. Several contextual keywords were added for module declarations. ([Oracle Docs](#))

Keyword	Meaning	Example
package	Defines package	package com.example;
import	Imports type/member	import java.util.List;
module	Defines module	module com.example.app {}

open	Declares open module	open module app {}
requires	Module dependency	requires java.sql;
transitive	Transitive dependency	requires transitive api;
exports	Exposes package	exports com.example.api;
opens	Opens package for reflection	opens com.example.model;
to	Qualified export/open	exports com.api to client;
uses	Declares service use	uses com.example.Service;
provides	Declares service provider	provides S with Impl;
with	Associates service with implementation	provides S with Impl;

Example:

```

module com.example.application {
    requires java.sql;

    exports com.example.api;

    opens com.example.model;

    uses com.example.PaymentService;

    provides com.example.PaymentService
        with com.example.DefaultPaymentService;
}

```

11. Primitive Types vs Keywords

It is important to distinguish **primitive-type keywords** from reference types.

Keyword	Primitive?	Example
boolean	Yes	<code>boolean b = true;</code>
byte	Yes	<code>byte b = 1;</code>
short	Yes	<code>short s = 10;</code>
int	Yes	<code>int i = 10;</code>
long	Yes	<code>long l = 10L;</code>
float	Yes	<code>float f = 10.0f;</code>
double	Yes	<code>double d = 10.0;</code>
char	Yes	<code>char c = 'A';</code>
String	No	<code>String s = "Java";</code>
Object	No	<code>Object o = new Object();</code>
Integer	No	<code>Integer i = 10;</code>

String, Object, Integer, ArrayList, etc. are classes, not Java keywords.

12. Important Special Cases

const

Java reserves const, but it has **no language construct using const**.

Incorrect:

```
const int MAX = 100;
```

Correct:

```
final int MAX = 100;
```

The JLS states that const and goto are reserved even though they are not currently used. ([Oracle Docs](#))

goto

Java does not provide a goto statement.

```
goto start; // compilation error
```

It remains reserved.

strictfp

Historically:

```
strictfp class Calculator {  
    // strict floating-point semantics
```

```
}
```

However, Java 26 considers strictfp **obsolete** and recommends that it not be used in new code. ([Oracle Docs](#))

13. var Is Not a Traditional Reserved Keyword

This is an especially important interview point.

```
var name = "Anurag";
```

var provides **local-variable type inference**.

The compiler determines:

```
var name = "Anurag";
```

as effectively:

```
String name = "Anurag";
```

But var does **not** mean dynamically typed variable.

For example:

```
var number = 100;  
number = "Java"; // compilation error
```

The variable remains statically typed as int.

The JLS classifies var as a **contextual keyword**, not a traditional reserved keyword. ([Oracle Docs](#))

14. record

A record provides a concise way to declare a class intended primarily to model data.

```
record Employee(int id, String name) {  
}
```

The compiler provides important members such as `accessors`, `equals`, `hashCode`, and `toString` according to the record semantics.

Example:

```
Employee e = new Employee(101, "Anurag");
```

```
System.out.println(e.id());  
System.out.println(e.name());
```

record became a standard Java language feature in Java 16 after earlier preview releases.

15. sealed, permits, and non-sealed

These keywords control inheritance.

Sealed class

```
sealed class Shape  
    permits Circle, Rectangle {  
}
```

Only the permitted direct subclasses can extend Shape.

```
final class Circle extends Shape {  
}
```

or:

```
non-sealed class Rectangle extends Shape {  
}
```

The three related concepts are:

Keyword	Purpose
sealed	Restricts direct inheritance
permits	Lists permitted direct subclasses
non-sealed	Allows unrestricted further inheritance from that permitted subclass

Java's sealed-type rules require permitted subclasses to explicitly establish how the hierarchy continues, typically using `final`, `sealed`, or `non-sealed`. ([Oracle Docs](#))

16. yield

`yield` is associated with **switch expressions**.

```
int result = switch (day) {  
    case 1 -> 100;  
    case 2 -> 200;  
    default -> {  
        yield 0;  
    }  
};
```

It differs from `return`:

return	yield
Returns from a method/lambda	Produces a value from a switch expression
Exits the enclosing method/lambda	Exits the switch expression's relevant block
return result;	yield result;

yield is a contextual keyword. ([Oracle Docs](#))

17. when

Java 26 includes when among its contextual keywords.

It is used as a **guard** in applicable pattern-matching constructs:

```
static String classify(Object obj) {
    return switch (obj) {
        case String s when !s.isEmpty() -> "Non-empty string";
        case String s -> "Empty string";
        default -> "Other";
    };
}
```

The Java 25/26 JLS recognizes when contextually in a Guard. ([Oracle Docs](#))

18. true, false, and null Are NOT Keywords

This is a common interview question.

Token	Actual classification	Example
true	Boolean literal	boolean b = true;
false	Boolean literal	boolean b = false;
null	Null literal	String s = null;

Therefore, technically:

true → literal

false → literal

null → literal

They are not members of the Java keyword list. ([Oracle Docs](#))

19. Evolution of Java Keywords — Java 1 → Java 26

The most useful historical view is to identify **when new language keywords/contextual keywords appeared**.

Java version	Keyword(s) / change	Feature
Java 1.0	Original keyword set	Core Java language
Java 1.1	No major new language keyword	Inner classes and other platform/language evolution did not require a new keyword

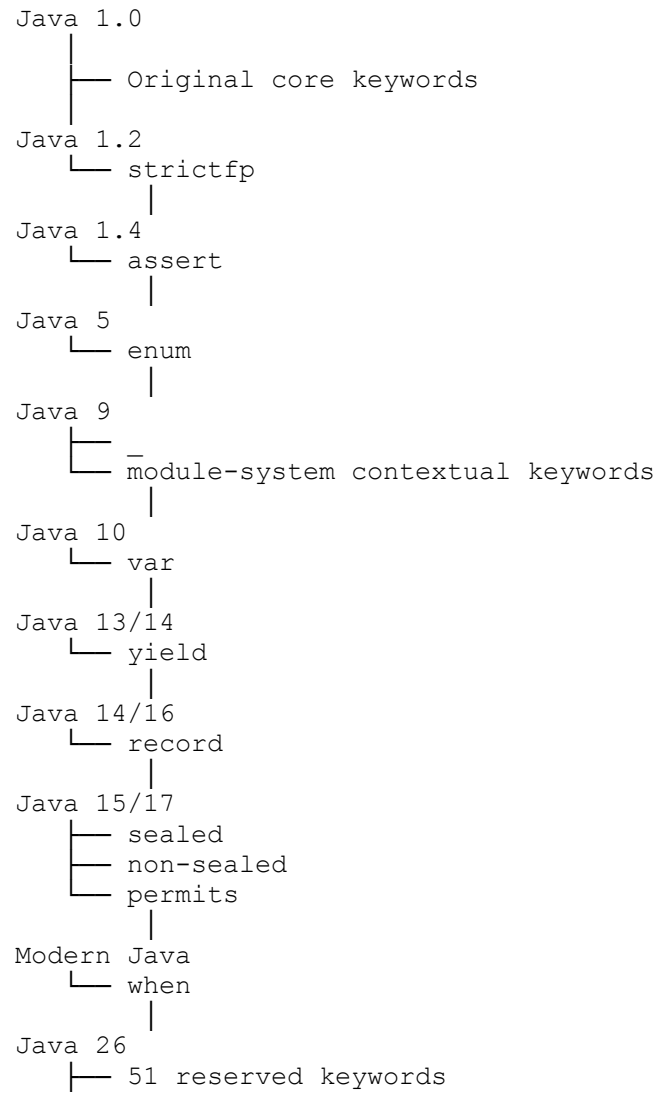
Java 1.2	strictfp	Strict floating-point semantics
Java 1.3	No new keyword	—
Java 1.4	assert	Assertion facility
Java 5	enum	Enumeration types
Java 6	No new keyword	—
Java 7	No new keyword	Multi-catch, try-with-resources, diamond, strings in switch, etc. used existing syntax
Java 8	No new keyword	Lambdas, method references, default/static interface methods
Java 9	_ becomes a reserved keyword; module-related contextual keywords	JPMS/module system
Java 10	var	Local-variable type inference
Java 11	No new keyword	—
Java 12	No new permanent keyword	Switch expressions preview
Java 13	yield introduced for switch expressions as a preview feature	Switch expressions
Java 14	yield and record continued/appeared through language evolution	Switch expressions and records
Java 15	sealed, non-sealed, permits preview	Sealed classes/interfaces
Java 16	record and related record syntax standardized; sealed features continued as preview	Records

Java 17	sealed, non-sealed, permits standardized	Sealed classes/interfaces
Java 18	No new keyword	—
Java 19	when appeared in pattern-switch preview specifications	Pattern matching evolution
Java 20	when continued in preview specifications	Pattern matching
Java 21	when not standardized as a permanent reserved keyword; pattern matching/switch finalized without treating it as a traditional reserved keyword	Pattern matching
Java 22	when continued as contextual syntax in language evolution	Pattern matching
Java 23	when remained contextual in relevant specifications	Pattern matching
Java 24	when appears in the JLS contextual-keyword set	Pattern matching/module evolution
Java 25	when is explicitly recognized as a contextual keyword in a Guard	Pattern matching
Java 26	51 reserved + 17 contextual keywords	Current JLS keyword model

The historical distinction is important: **a feature can be introduced as a preview feature before its syntax becomes final**, so "first appeared" and "became permanent" are not always the same Java release. For example, records and sealed types went through preview stages before becoming standard. ([Oracle Docs](#))

20. Java 1 → Java 26 Keyword Growth

A simplified evolution looks like this:



21. Reserved vs Contextual Keywords

Parameter	Reserved keyword	Contextual keyword
Meaning	Always has keyword status	Has keyword meaning only in specific contexts
Can generally be an identifier?	No	Some can be interpreted as identifiers outside their special contexts
Examples	class, public, int, return	var, record, module, when
Main reason	Core language syntax	New syntax while preserving compatibility
Java 26 count	51	17
Example	class Student {}	var x = 10;

The JLS explicitly says that an identifier cannot have the same spelling as a reserved keyword, while a contextual keyword can potentially have identifier interpretation depending on context. ([Oracle Docs](#))

22. Complete Java 26 Keyword Cheat Sheet

Reserved keywords — 51

abstract
assert
boolean
break
byte
case

catch
char
class
const
continue
default
do
double
else
enum
extends
final
finally
float
for
goto
if
implements
import
instanceof
int
interface
long
native
new
package
private
protected
public
return
short
static
strictfp
super
switch
synchronized

this
throw
throws
transient
try
void
volatile
while
-

Contextual keywords — 17

exports
opens
requires
uses
yield
module
permits
sealed
var
non-sealed
provides
to
when
open
record
transitive
with

These lists correspond to the Java SE 26 JLS §3.9. ([Oracle Docs](#))

23. The Most Important Interview Distinctions

Question	Correct answer
How many reserved keywords does Java 26 have?	51
How many contextual keywords does Java 26 have?	17
Total keyword spellings recognized by the JLS?	68
Is String a keyword?	No
Is Object a keyword?	No
Is true a keyword?	No — boolean literal
Is false a keyword?	No — boolean literal
Is null a keyword?	No — null literal
Is var a reserved keyword?	No — contextual keyword
Is record a reserved keyword?	No — contextual keyword
Is sealed a reserved keyword?	No — contextual keyword
Is yield a reserved keyword?	No — contextual keyword
Is when a reserved keyword?	No — contextual keyword
Is goto supported?	No; reserved but unused
Is const supported?	No; reserved but unused
Is strictfp still recommended?	No; obsolete in modern Java

Can _ be used as a normal identifier?	No
Which version introduced assert?	Java 1.4
Which version introduced enum?	Java 5
Which version introduced modules?	Java 9
Which version introduced var?	Java 10
Which version introduced switch-expression yield?	Preview in Java 13; standardized in Java 14
Which version standardized records?	Java 16
Which version standardized sealed types?	Java 17

Oracle's current JLS confirms the 51/17 classification, the special status of `_`, the obsolete status of `strictfp`, and the fact that `true`, `false`, and `null` are literals rather than keywords. ([Oracle Docs](#))

Official reference

[Java SE 26 — Java Language Specification](#) — the authoritative specification for the current Java language. ([Oracle Docs](#))